

WEEK 4 INTERNSHIP ASSIGNMENT

FILE HANDLING, EXCEPTION HANDLING, PYTHON LIBRARIES AND MINI PYTHON PROJECT

Submitted By:-

Name: Amrutha B V

Course: Vth Semester BCA

Internship: Python Programming

Internship Week: 4

ACKNOWLEDGEMENT:-

I sincerely thank my internship mentors and faculty members for providing me with the opportunity to complete this Week 4 Python programming assignment.

This assignment helped me improve my practical knowledge of file handling, exception handling and Python libraries. It also gave me an opportunity to develop a Mini Python Project by applying the concepts learned during the previous weeks.

I am grateful for the guidance and support provided throughout the internship. This assignment has helped me improve my programming, problem-solving and project development skills.

INTRODUCTION:-

Python is a powerful and easy-to-learn programming language that provides many features for developing useful applications. In addition to basic programming concepts, Python provides facilities for handling files, managing runtime errors and using built-in libraries.

File handling allows programs to create, read, write and update files. Exception handling helps programs manage errors without suddenly terminating. Python libraries provide ready-to-use functions for performing different operations.

In this week's assignment, these concepts are applied through three individual programs and a Mini Python Project. The project combines the concepts learned during Weeks 1–4 into a functional and user-friendly application.

OBJECTIVE:-

The main objectives of this assignment are:

- To understand basic file handling operations in Python.
- To learn how to create, read, write and append files.
- To understand exception handling using try and except.
- To handle common runtime errors.

- To understand the use of Python's built-in libraries.
- To use the Random, Math and Datetime libraries.
- To apply programming concepts in a Mini Python Project.
- To improve problem-solving and programming skills.
- To develop a functional and user-friendly Python application.

TOPICS COVERED:-

The following topics are covered in this assignment:

- 1.File Handling
- 2.Reading and Writing Files
- 3.Exception Handling
- 4.Random Library
- 5.Math Library
- 6.Datetime Library
- 7.Mini Python Project
- 8.Project Development
- 9.Variables and Data Types
- 10.Input and Output
- 11.Conditional Statements
- 12.Loops
- 13.Functions
- 14.Data Structures
- 15.File Handling in a Project
- 16.Exception Handling in a Project

TASK 1: FILE HANDLING

Aim:-

To create a Python program that performs basic file operations such as creating, writing, reading, appending and displaying updated file content.

Introduction:-

File handling is an important feature of Python that allows programs to store and retrieve information from files. Python provides the `open()` function to work with files.

Different file modes are available for performing different operations. The `w` mode is used for writing, `r` is used for reading and `a` is used for appending data.

Requirements:-

The program should:

- Create a text file.
- Write data into the file.
- Read and display the data.
- Append new data to the existing file.
- Display the updated file content.

•Concepts Used

- File opening
- File writing
- File reading
- File appending
- File closing

Program Explanation:-

First, the program creates a text file and writes initial information into it. The file is then opened in read mode to display its contents.

Next, new information is appended to the existing file. Finally, the file is opened again and the updated contents are displayed.

Source Code:-

task1_file_handling.py



new*

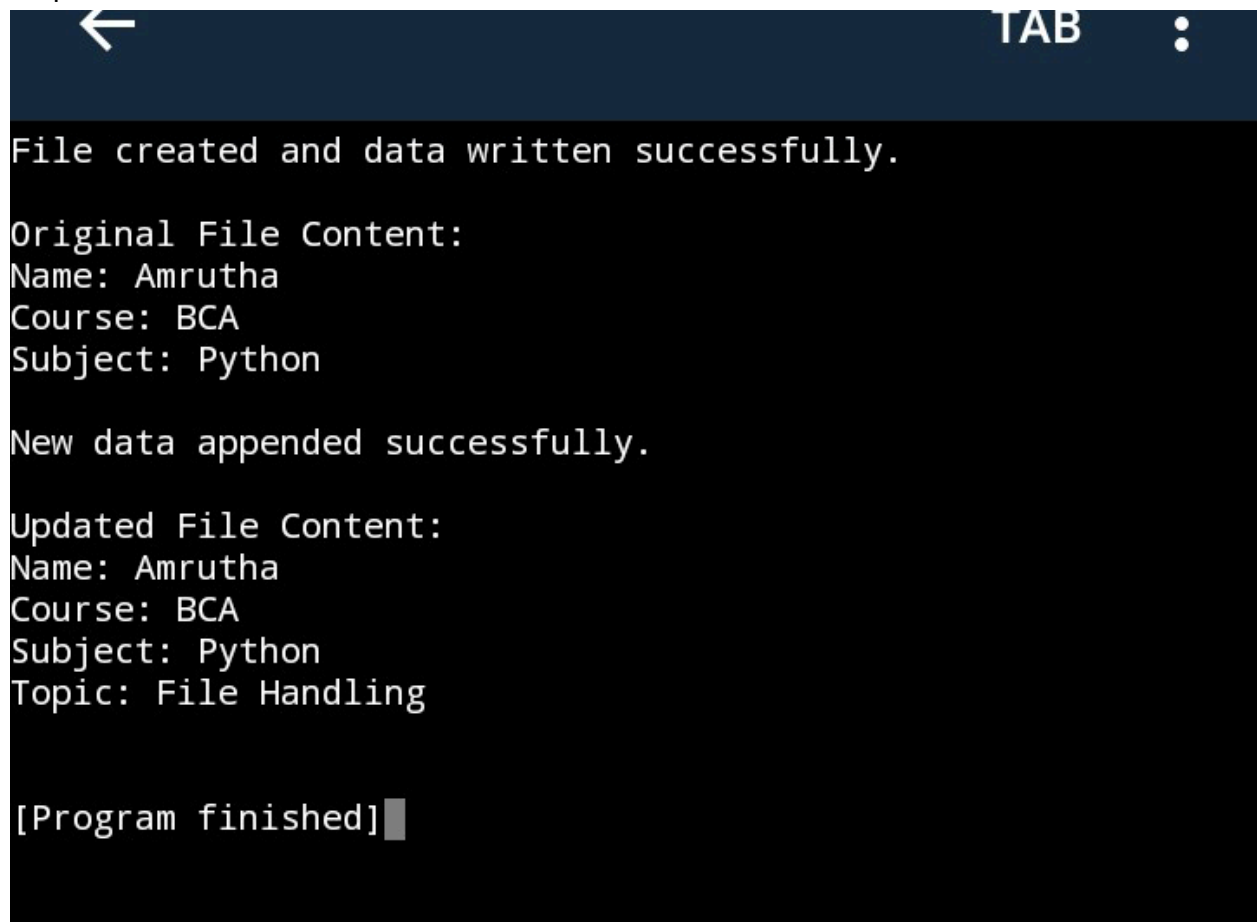


```
1  # Task 1: File Handling
2
3  # Create a file and write data
4  with open("student.txt", "w") as file:
5      file.write("Name: Amrutha\n")
6      file.write("Course: BCA\n")
7      file.write("Subject: Python\n")
8
9  print("File created and data written
    successfully.")
10
11 # Read and display the file
12 with open("student.txt", "r") as file:
13     print("\nOriginal File Content:")
14     print(file.read())
15
16 # Append new data
17 with open("student.txt", "a") as file:
18     file.write("Topic: File Handling\n")
19
20 print("New data appended successfully.")
21
22 # Display updated content
23 with open("student.txt", "r") as file:
24     print("\nUpdated File Content:")
25     print(file.read())
```

Output:-

The output displays the original file content and the updated content after appending new data.

Output Screenshot:-



```
← TAB :  
File created and data written successfully.  
Original File Content:  
Name: Amrutha  
Course: BCA  
Subject: Python  
New data appended successfully.  
Updated File Content:  
Name: Amrutha  
Course: BCA  
Subject: Python  
Topic: File Handling  
[Program finished]█
```

Learning Outcome:-

After completing this task, I learned how to create, write, read and append data to a text file using Python. I also understood the importance of opening and closing files correctly.

TASK 2: EXCEPTION HANDLING:-**Aim**

To create a Python program that handles common runtime errors using appropriate try and except blocks.

Introduction:-

Exception handling is used to handle runtime errors in a program. Without exception handling, an unexpected error may terminate the program.

Python provides try and except blocks to detect and handle errors. This makes programs more reliable and user-friendly.

Requirements:-

The program demonstrates handling of:

- Invalid user input
- Division by zero
- File not found

•Concepts Used

- try
- except
- ValueError
- ZeroDivisionError
- FileNotFoundError

Program Explanation:-

The program uses separate try and except blocks to handle different types of runtime errors.

If the user enters invalid numeric data, a ValueError is handled. If a number is divided by zero, a ZeroDivisionError is handled. When a file does not exist, a FileNotFoundError is handled.

Meaningful error messages are displayed instead of allowing the program to terminate unexpectedly.

Source Code:-



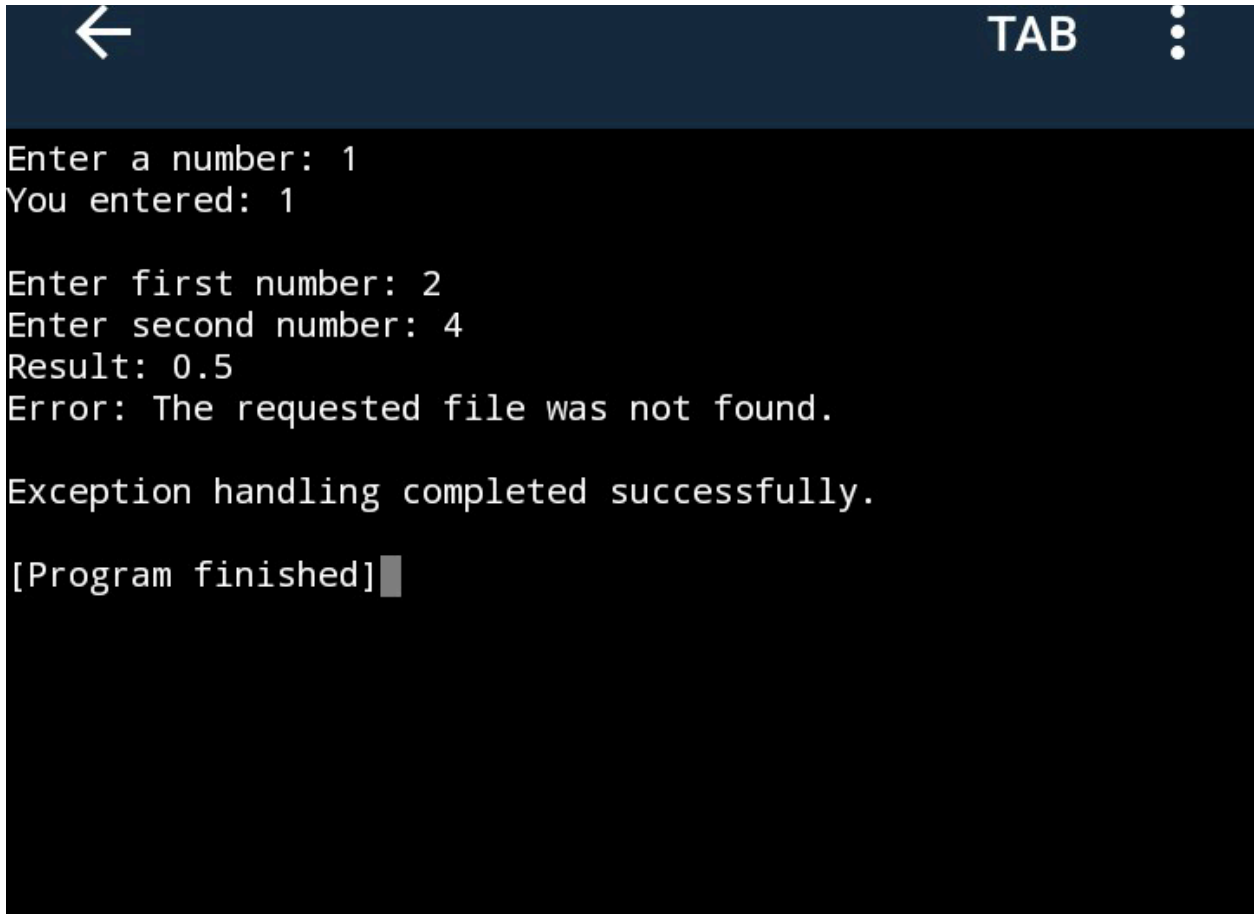
new*



```
3 # Handling invalid user input
4 try:
5     number = int(input("Enter a number: "))
6     print("You entered:", number)
7 except ValueError:
8     print("Error: Please enter a valid number.")
9
10 # Handling division by zero
11 try:
12     a = int(input("\nEnter first number: "))
13     b = int(input("Enter second number: "))
14     result = a / b
15     print("Result:", result)
16 except ZeroDivisionError:
17     print("Error: Division by zero is not allowed.")
18 except ValueError:
19     print("Error: Please enter valid numbers.")
20
21 # Handling file not found
22 try:
23     with open("missing_file.txt", "r") as file:
24         print(file.read())
25 except FileNotFoundError:
26     print("Error: The requested file was not found.")
```

Output:-

The program displays appropriate messages for invalid input, division by zero and a missing file.
Output Screenshot

A screenshot of a terminal window with a dark blue header bar containing a back arrow, the word 'TAB', and a vertical ellipsis. The terminal text is as follows:

```
Enter a number: 1
You entered: 1

Enter first number: 2
Enter second number: 4
Result: 0.5
Error: The requested file was not found.

Exception handling completed successfully.

[Program finished]
```

Learning Outcome:-

After completing this task, I learned how to identify and handle common runtime errors using Python exception handling. I also learned how meaningful error messages can make programs more reliable and user-friendly.

TASK 3: PYTHON LIBRARIES

Aim:-

To demonstrate the use of Python's Random, Math and Datetime libraries.

Introduction:-

Python libraries contain predefined functions and modules that make programming easier. They allow programmers to perform common operations without writing everything from scratch.

This task demonstrates three built-in Python libraries: Random, Math and Datetime.

1. Random Library

The Random library is used to generate random values.
In this program, it is used to generate a random number.

2. Math Library

The Math library provides mathematical functions.
In this program, it is used to calculate the square root and power of a number.

3. Datetime Library

The Datetime library is used to work with dates and times.
In this program, it is used to display the current date and time.

Source Code

The source code is provided in:



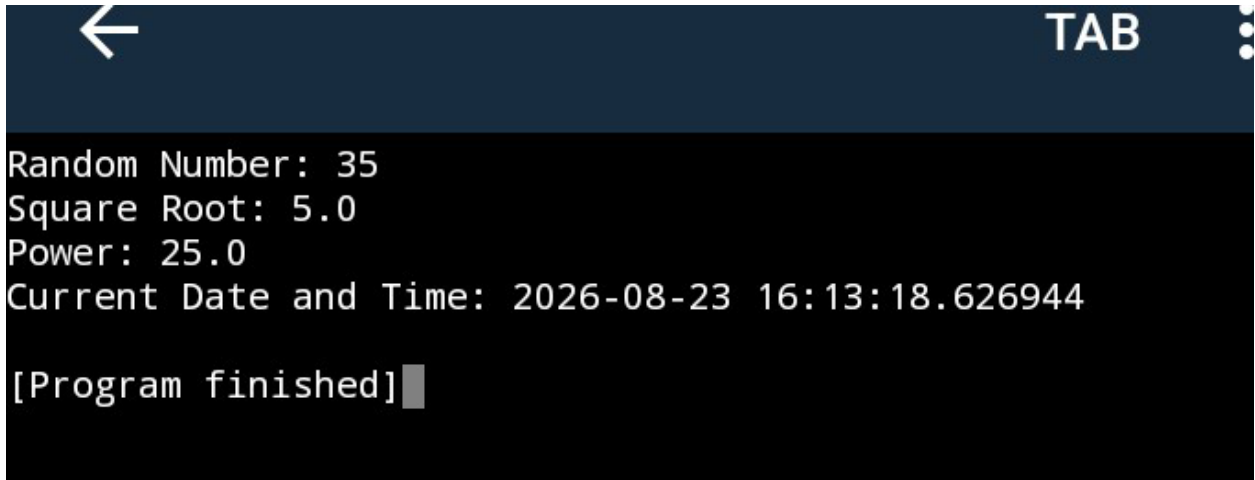
new*



```
1  # Task 3: Python Libraries
2
3  import random
4  import math
5  from datetime import datetime
6
7  # Random library:
8  # Used to generate a random number.
9  random_number = random.randint(1, 100)
10 print("Random Number:",
      random_number)
11
12 # Math library:
13 # Used for mathematical operations.
14 number = 25
15 print("Square Root:", math.sqrt(number))
16 print("Power:", math.pow(5, 2))
17
18 # Datetime library:
19 # Used to display the current date and
   time.
20 current_time = datetime.now()
21 print("Current Date and Time:",
      current_time)
```

Output:-

The program displays a random number, mathematical results and the current date and time.
Output Screenshot

A screenshot of a terminal window with a dark blue header bar containing a white back arrow on the left and the word 'TAB' on the right. The terminal content is as follows:

```
Random Number: 35
Square Root: 5.0
Power: 25.0
Current Date and Time: 2026-08-23 16:13:18.626944

[Program finished]
```

Learning Outcome:-

After completing this task, I learned how built-in Python libraries can simplify programming. I understood the practical uses of Random, Math and Datetime libraries.

TASK 4: MINI PYTHON PROJECT EXPENSE TRACKER

Project Title:-

Expense Tracker Using Python

Introduction:-

An Expense Tracker is a simple application used to record and manage daily expenses. It allows users to add expenses, view recorded expenses, calculate the total amount and delete an expense.

The project applies the Python concepts learned during Weeks 1–4. It uses variables, data types, input and output, conditional statements, loops, functions, data structures, file handling, exception handling and a Python library.

OBJECTIVE OF THE PROJECT:-

The main objectives of the Expense Tracker are:

- To record daily expenses.
- To store expense details in an organized manner.
- To display saved expenses.

- To calculate the total expense.
- To delete an unwanted expense.
- To store expense data in a file.
- To handle invalid user input.
- To provide a simple and user-friendly interface.
- To apply Python programming concepts in a practical project.

PROBLEM STATEMENT:-

Managing daily expenses manually can be difficult when many transactions are involved. It may become difficult to remember individual expenses and calculate the total amount.

The Expense Tracker provides a simple solution by allowing users to add, view, calculate and delete expenses. The application stores the data in a file so that the information can be maintained for future use.

FEATURES:-

The main features of the Expense Tracker are:

- 1.Add a new expense.
- 2.View all recorded expenses.
- 3.Calculate the total expense.
- 4.Delete an expense.
- 5.Save expense information in a file.
- 6.Load existing expense information.
- 7.Handle invalid input.
- 8.Display the current date for each expense.
- 9.Provide a simple menu-driven interface.
- 10.Exit the application safely.

CONCEPTS USED:-

The project uses the following Python concepts:

1. Variables and Data Types:-

Variables are used to store expense amounts, categories, descriptions and dates. Different data types such as strings, integers and floating-point numbers are used.

2. Input and Output:-

The input() function is used to collect information from the user and the print() function is used to display results.

3. Conditional Statements:-

if, elif and else statements are used to perform different operations according to the user's menu choice.

4. Loops:-

A while loop is used to repeatedly display the menu until the user chooses to exit.

5. Functions:-

Functions are used to divide the program into smaller and reusable sections such as adding, viewing and deleting expenses.

6. Data Structures:-

A list of dictionaries is used to store multiple expense records in an organized form.

7. File Handling:-

A text file is used to save expense information and retrieve it when the program starts.

8. Exception Handling:-

try and except blocks are used to handle invalid amounts and file-related errors.

9. Python Library:-

The datetime library is used to obtain the current date when an expense is added.

PROJECT WORKING:-

When the program starts, it displays a menu containing different options.

The user can select an option by entering the corresponding number.

If the user selects Add Expense, the program asks for the amount, category and description.

If View Expenses is selected, all saved expenses are displayed.

The Calculate Total option calculates and displays the total amount of all expenses.

The Delete Expense option removes a selected expense.

The Exit option safely terminates the program.

Expense data is stored in a text file so that the information can be retained.

SOURCE CODE:-

```
1 # Mini Python Project: Expense Tracker
2
3 from datetime import datetime
4
5 FILE_NAME = "expenses.txt"
6
7
8 # Load expenses from the file
9 def load_expenses():
10     expenses = []
11
12     try:
13         with open(FILE_NAME, "r") as file:
14             for line in file:
15                 data = line.strip().split("|")
16
17                 if len(data) == 4:
18                     expense = {
19                         "date": data[0],
20                         "category": data[1],
21                         "amount": float(data[2]),
22                         "description": data[3]
23                     }
24                     expenses.append(expense)
25
26     except FileNotFoundError:
```

```

32 # Save expenses to the file
33 def save_expenses(expenses):
34     with open(FILE_NAME, "w") as file:
35         for expense in expenses:
36             file.write(
37                 f"{expense['date']}|"
38                 f"{expense['category']}|"
39                 f"{expense['amount']}|"
40                 f"{expense['description']}\n"
41             )
42
43
44 # Add a new expense
45 def add_expense(expenses):
46     try:
47         amount = float(input("Enter amount:
48 "))
49         if amount <= 0:
50             print("Amount must be greater than
51             zero.")
52             return
53         category = input("Enter category: ")
54         description = input("Enter
55         description: ")
56         date = datetime.now().strftime("%Y-

```



```
58     expense = {
59         "date": date,
60         "category": category,
61         "amount": amount,
62         "description": description
63     }
64
65     expenses.append(expense)
66     save_expenses(expenses)
67
68     print("Expense added successfully.")
69
70     except ValueError:
71         print("Error: Please enter a valid
72 amount.")
73
74 # View all expenses
75 def view_expenses(expenses):
76     if not expenses:
77         print("No expenses found.")
78         return
79
80     print("\n----- ALL EXPENSES -----")
81
82     for i, expense in enumerate(expenses,
83 start=1):
84         print(
```



```
88     )
89
90
91     # Calculate total expenses
92     def calculate_total(expenses):
93         total = sum(expense["amount"] for
expense in expenses)
94
95         print(f"\nTotal Expense: ₹{total:.2f}")
96
97
98     # Delete an expense
99     def delete_expense(expenses):
100         view_expenses(expenses)
101
102         if not expenses:
103             return
104
105         try:
106             number = int(input("Enter expense
number to delete: "))
107
108             if 1 <= number <= len(expenses):
109                 removed = expenses.pop(number -
1)
110                 save_expenses(expenses)
111
112                 print(
113                     f"Expense
```

```
121
122
123 # Main program
124 def main():
125     expenses = load_expenses()
126
127     while True:
128
129         print("\n=====
130         print("      EXPENSE TRACKER")
131
132         print("=====
133         print("1. Add Expense")
134         print("2. View Expenses")
135         print("3. Calculate Total")
136         print("4. Delete Expense")
137         print("5. Exit")
138
139         choice = input("Enter your choice: ")
140
141         if choice == "1":
142             add_expense(expenses)
143
144         elif choice == "2":
145             view_expenses(expenses)
146
147         elif choice == "3":
```

```
139     if choice == "1":
140         add_expense(expenses)
141
142     elif choice == "2":
143         view_expenses(expenses)
144
145     elif choice == "3":
146         calculate_total(expenses)
147
148     elif choice == "4":
149         delete_expense(expenses)
150
151     elif choice == "5":
152         print("Thank you for using Expense
Tracker.")
153         break
154
155     else:
156         print("Invalid choice. Please select
1 to 5.")
157
158
159 # Start the program
160 if __name__ == "__main__":
161     main()
```

OUTPUT SCREENSHOTS:-

The following screenshots should be included in the project report:

- Main Menu
- Adding an Expense
- Viewing Expenses
- Total Expense
- Deleting an Expense
- Invalid Input/Error Handling
- Exit Message

```
=====
```

EXPENSE TRACKER

```
=====
```

1. Add Expense
2. View Expenses
3. Calculate Total
4. Delete Expense
5. Exit

Enter your choice: 1

Enter amount: 250

Enter category: food

Enter description: lunch

Expense added successfully.

```
=====
```

EXPENSE TRACKER

```
=====
```

1. Add Expense
2. View Expenses
3. Calculate Total
4. Delete Expense
5. Exit

Enter your choice: 2

----- ALL EXPENSES -----

1. 2026-08-23 | food | ₹250.00 | lunch

```
=====
```

EXPENSE TRACKER

```
=====
```

1. Add Expense
2. View Expenses
3. Calculate Total
4. Delete Expense
5. Exit

Enter your choice: 3

Total Expense: ₹250.00

```
=====
```

EXPENSE TRACKER

```
=====
```

1. Add Expense

3. Calculate Total

4. Delete Expense

5. Exit

Enter your choice: 2

----- ALL EXPENSES -----

1. 2026-08-23 | food | ₹250.00 | lunch

=====

EXPENSE TRACKER

=====

1. Add Expense

2. View Expenses

3. Calculate Total

4. Delete Expense

5. Exit

Enter your choice: 3

Total Expense: ₹250.00

=====

EXPENSE TRACKER

=====

1. Add Expense

2. View Expenses

3. Calculate Total

4. Delete Expense

5. Exit

Enter your choice: 4

----- ALL EXPENSES -----

1. 2026-08-23 | food | ₹250.00 | lunch

Enter expense number to delete: 1

Expense 'lunch' deleted successfully.

=====

EXPENSE TRACKER

=====

1. Add Expense

2. View Expenses

3. Calculate Total

4. Delete Expense

5. Exit

Enter your choice: 8

PROJECT EXPLANATION:-

The Expense Tracker follows a menu-driven approach.

The program first loads previously saved expense information from the file. It then displays the main menu.

The Add Expense function collects the amount, category and description from the user. The current date is automatically obtained using the Datetime library.

The View Expenses function displays all stored records.

The Calculate Total function adds all expense amounts and displays the total.

The Delete Expense function allows the user to remove an expense record.

File handling is used to save and retrieve information. Exception handling is used to prevent the program from terminating because of invalid input or file errors.

CHALLENGES FACED:-

During the development of the project, some challenges were encountered.

1. Handling invalid numerical input was a challenge.
2. Managing expense records in a structured format required proper use of data structures.
3. Reading and writing data to a file required understanding of file modes.
4. Handling missing or incorrect file data required exception handling.
5. Designing a simple menu-driven program required proper use of loops and conditional statements.

These challenges helped improve my programming and debugging skills.

CODE QUALITY:-

The project was developed using:

- Meaningful variable names.
- Separate functions for different operations.
- Comments for important sections.
- Proper indentation.
- Exception handling.

- Simple and user-friendly menu options.
- Organized program structure.

These practices make the program easier to understand, maintain and modify.

OVERALL LEARNING OUTCOME:-

After completing this assignment, I gained practical knowledge of Python file handling, exception handling and built-in libraries.

I learned how to create, read, write and append files. I also learned how to handle runtime errors using try and except blocks.

The Mini Python Project helped me combine different Python concepts such as variables, data types, conditions, loops, functions, data structures, file handling, exception handling and libraries into one functional application.

This assignment improved my confidence in Python programming and helped me understand how programming concepts can be applied to solve real-world problems.

CONCLUSION:-

This Week 4 assignment provided practical experience with important Python programming concepts.

The File Handling task helped me understand how data can be stored and managed using files. The Exception Handling task taught me how to handle runtime errors safely. The Python Libraries task demonstrated how built-in modules can simplify programming.

The Expense Tracker project allowed me to combine the concepts learned during Weeks 1–4 into a single functional application. The project provides a simple way to record, view, calculate and manage expenses.

Overall, this assignment strengthened my Python programming, problem-solving, debugging and project development skills.